

relatorio

July 6, 2026

Universidade de Brasília
Departamento de Engenharia Elétrica

Comunicações Móveis - PPGEE UnB

Trabalho 3 - Cell Free Massive MIMO

Professor: Higo Thaian Pereira da Silva

Aluno: Artur Padovesi Piratelli

Matrícula: 261101032

Brasília
2026

1 Introdução

As redes **Cell-Free Massive MIMO** representam uma evolução das arquiteturas celulares tradicionais ao substituir estações rádio-base de alta potência por um grande número de Pontos de Acesso (Access Points - APs) distribuídos geograficamente e conectados a uma Unidade Central de Processamento (Central Processing Unit - CPU). Nessa arquitetura, todos os APs cooperam para atender simultaneamente todos os Equipamentos de Usuário (User Equipments - UEs), compartilhando os mesmos recursos de tempo e frequência.

Neste notebook é implementado um simulador de uma rede Cell-Free Massive MIMO baseado no modelo proposto por Ngo *et al.* e nos slides sobre Massive MIMO da disciplina de Comunicações Móveis do professor Higo Thaian. O objetivo é avaliar o desempenho do sistema por meio de simulações de Monte Carlo, considerando tanto a variabilidade da geometria da rede quanto as diferentes realizações do canal de pequena escala.

O modelo implementado considera:

- desvanecimento em larga escala utilizando o modelo **Close-In Free-Space (CI)** com sombreamento log-normal;
- desvanecimento Rayleigh para representar o canal de pequena escala;
- estimação de canal por **MMSE** utilizando pilotos ortogonais;
- transmissão em downlink utilizando **Conjugate Beamforming**.

Duas métricas principais de desempenho são analisadas:

- **SINR (Signal-to-Interference-plus-Noise Ratio)**;
- **Taxa alcançável (Achievable Rate)**, obtida a partir da capacidade de Shannon.

Para cada geometria da rede são geradas diversas realizações independentes do canal de pequena escala, mantendo fixas as posições dos APs e UEs. Essa separação permite avaliar isoladamente os efeitos do desvanecimento rápido e da propagação em larga escala.

Ao longo deste notebook serão investigados os impactos de diferentes parâmetros do sistema sobre o desempenho da rede, por meio da comparação das distribuições empíricas (ECDFs) da SINR e da taxa alcançável. Os parâmetros analisados neste documento são:

- número de APs;
- densidade da implantação da rede;
- número de usuários;
- comprimento da sequência de pilotos;
- potência de transmissão.

Sempre que não explicitado o contrário, será utilizada uma configuração de referência (*baseline*), a partir da qual cada parâmetro será variado individualmente para avaliar sua influência sobre o desempenho do sistema. O *baseline* pode ser visto na tabela 1.

Tabela 1 – Parâmetros utilizados na configuração de referência.

Parâmetro	Valor
Frequência da portadora (f_c)	3 GHz
Largura de banda (B)	20 MHz
Potência de transmissão dos APs (P_d)	200 mW

Parâmetro	Valor
Potência de transmissão dos pilotos (P_p)	100 mW
Figura de ruído	9 dB
Temperatura de ruído	296,15 K
Área de simulação	1000×1000 m
Altura dos APs	15 m
Altura dos UEs	1,65 m
Número de APs (M)	100
Número de UEs (K)	10
Comprimento da sequência de pilotos (τ_{cf})	10

2 Implementação

Em geral, a implementação segue exatamente o modelo proposto pelo professor Higo Thaian, com adoções do modelo original quando adequado. Estes casos foram: - Ω é ganho de larga escala do canal. As equações (29), (30) e (32) dos slides não considera a conversão de perda de percurso para ganho de percurso; - na equação (36) do documento γ é proporcional a $\sqrt{P_d}$, enquanto no modelo original de Ngo, na equação (8), γ é proporcional a $\sqrt{P_p}$. A modelagem original foi considerada; - A equação (21) do documento mostra uma versão de SINR diferente do artigo original. A versão original é utilizada no artigo de Ngo, na equação (26), e pode ser escrita da seguinte maneira, seguindo a notação do documento do professor:

$$\text{SINR}_k = \frac{P_d \left| \sum_{m=1}^M \sqrt{\eta_{m,k}} g_{m,k} \hat{g}_{m,k}^* \right|^2}{P_d \sum_{k' \neq k}^K \left| \sum_{m=1}^M \sqrt{\eta_{m,k'}} g_{m,k'} \hat{g}_{m,k'}^* \right|^2 + \sigma_w^2}. \quad (1)$$

3 Resultados

```
[1]: from cell_free import CellFree, plot_scenarios
import matplotlib.pyplot as plt
import numpy as np

basic_params = {
    "fc": 3e9, "bw": 20e6,
    "noise_fig_db": 9, "noise_temperature": 296.15,
    "area_dimensions": (1e3, 1e3),
    "ap_h": 15, "ue_h": 1.65,
    "num_ue": 10, "tau_cf": 10,
    "num_ap": 100,
    "pilot_pow_db": 10 * np.log10(0.1),
    "downlink_pow_db": 10 * np.log10(0.2),
}
```

3.1 Cenário 1 - Variando o Número de APs

Tabela 2 – Parâmetros utilizados no Cenário 1.

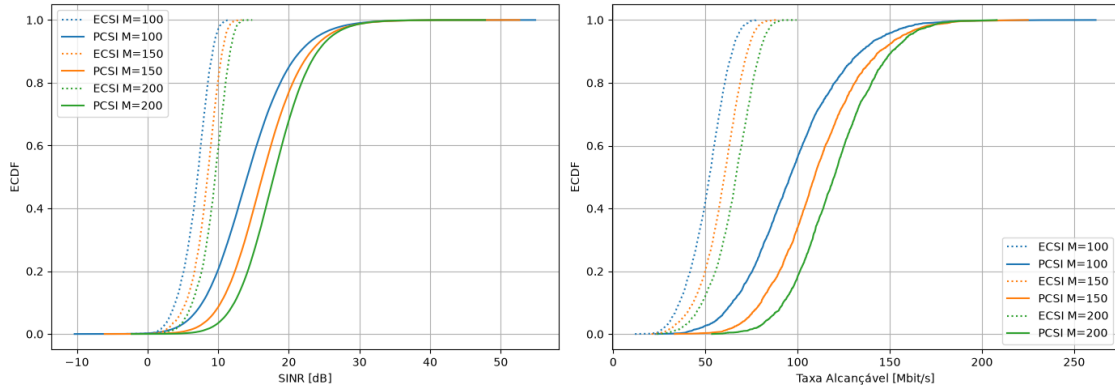
Parâmetro	Valor
Número de APs (M)	100, 150, 200

```
[2]: all_params = [
    {**basic_params, 'num_ap': x} for x in [100, 150, 200]
]
results = []
for par in all_params:
    cell_free_sim = CellFree(**par)
    rng = np.random.default_rng(12331)
    res = cell_free_sim.simulate(rng, 300, 100)
    label = f"M={par['num_ap']}"
    results.append((label, res))

plot_scenarios(results)

plt.show()
```

```
100%|      | 300/300 [00:03<00:00, 79.11it/s]
100%|      | 300/300 [00:05<00:00, 59.43it/s]
100%|      | 300/300 [00:06<00:00, 43.58it/s]
```



Como esperado, conforme M aumenta, a potência disponível para cada usuário aumenta, levando a uma melhora nas SINR e taxa alcançável.

3.2 Cenário 2 - Variando Número de Usuários

Foram utilizados diferentes números de usuários $K \in \{10, 20, 40\}$, mantendo sempre $\tau_{cf} = K$.

Tabela 3 – Parâmetros utilizados no Cenário 2.

Parâmetro	Valor
Número de UEs (K)	10, 20, 40, 100

```
[9]: all_params = [
    {**basic_params, "num_ue": 10, "tau_cf": 10},
    {**basic_params, "num_ue": 20, "tau_cf": 20},
    {**basic_params, "num_ue": 40, "tau_cf": 40},
    {**basic_params, "num_ue": 100, "tau_cf": 100},
]

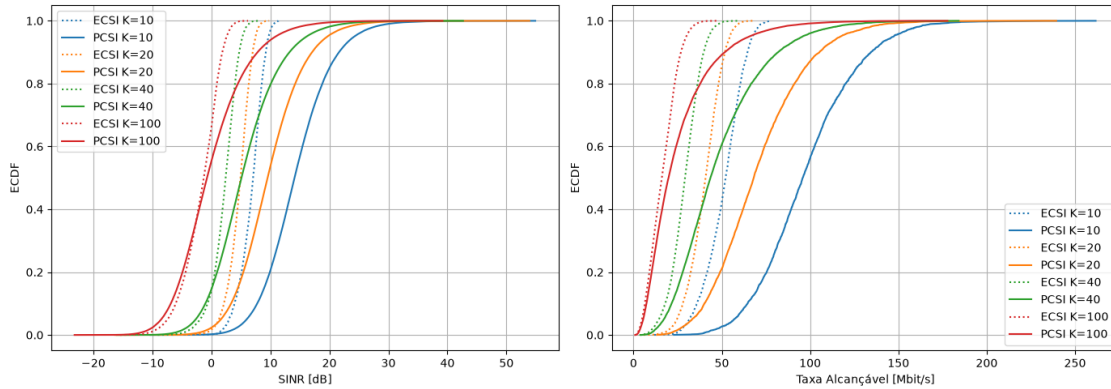
results = []

for par in all_params:
    cell_free_sim = CellFree(**par)
    rng = np.random.default_rng(12331)
    res = cell_free_sim.simulate(rng, 300, 100)

    label = f"K={par['num_ue']}"
    results.append((label, res))

plot_scenarios(results)
plt.show()
```

```
100%|      | 300/300 [00:03<00:00, 80.45it/s]
100%|      | 300/300 [00:07<00:00, 37.71it/s]
100%|      | 300/300 [00:24<00:00, 12.49it/s]
100%|      | 300/300 [02:48<00:00,  1.78it/s]
```



Mesmo que os pilotos se mantenham ortogonais, é possível notar que há uma piora considerável em SINR e taxa alcançável conforme K aumenta. Além disso, a piora é mais severa para a distribuição de SINR instantânea e taxa alcançável dado CSI perfeito do que para SINR e taxa alcançável estatísticos.

No caso particular com $K = 100$, percebe-se que, por 60% do tempo a taxa alcançável com CSI Perfeito não apresenta melhora significativa comparando com CSI estatística.

3.3 Cenário 3 - Variando o Número de APs e Área de Cobertura

Com o propósito de manter a densidade aproximadamente constante, a área de cobertura é aumentada conforme M aumenta.

Tabela 4 - Parâmetros utilizados no Cenário 3.

Parâmetro	Valor
Densidade de APs	100 por km ²
Número de APs (M)	100, 150, 200
Área de Cobertura	1×1 , 1.225×1.225 , 1.414×1.414 km

```
[11]: all_params = [
    {**basic_params, "num_ap": 100, "area_dimensions": (1000, 1000)},
    {**basic_params, "num_ap": 150, "area_dimensions": (1225, 1225)},
    {**basic_params, "num_ap": 200, "area_dimensions": (1414, 1414)},
]

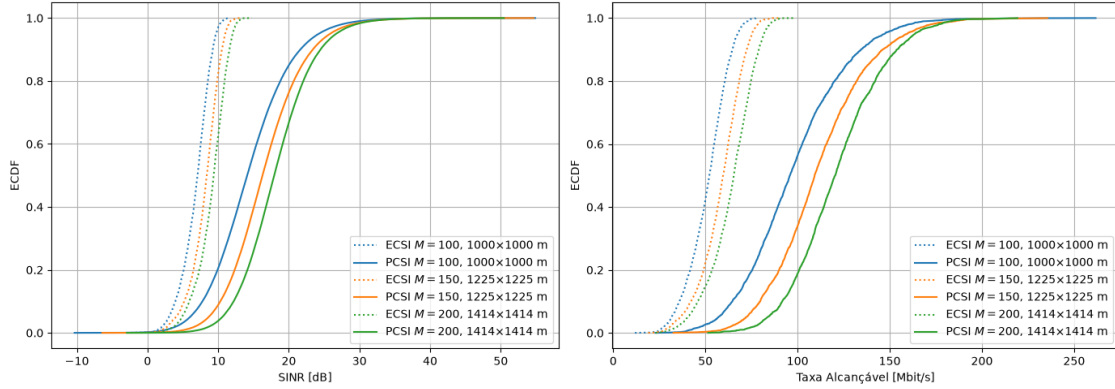
results = []

for par in all_params:
    cell_free_sim = CellFree(**par)
    rng = np.random.default_rng(12331)
    res = cell_free_sim.simulate(rng, 300, 100)

    label = (
        f"$M={par['num_ap']}$, "
        f"{par['area_dimensions'][0]:.0f}×{par['area_dimensions'][1]:.0f} m"
    )
    results.append((label, res))

plot_scenarios(results)
plt.show()
```

```
100%|      | 300/300 [00:03<00:00, 82.10it/s]
100%|      | 300/300 [00:04<00:00, 60.80it/s]
100%|      | 300/300 [00:06<00:00, 47.75it/s]
```



Diferente do esperado intuitivamente, os resultados do Cenário 3 estão praticamente os mesmos do Cenário 1. Ou seja, um aumento no número de APs no cenário base surte um efeito muito mais significativo que a sua diluição em área.

3.4 Cenário 4 - Variando o Número de APs e UEs

Com o propósito de manter a razão M/K constante, o número de usuários aumenta com mesma proporção com que M aumenta.

Tabela 5 - Parâmetros utilizados no Cenário 4.

Parâmetro	Valor
Razão M/K	10
Número de APs (M)	20, 100, 150, 200
Número de UEs	2, 10, 15, 20

```
[16]: all_params = [
    {**basic_params, "num_ap": 20, "num_ue": 2, "tau_cf": 2},
    {**basic_params, "num_ap": 100, "num_ue": 10, "tau_cf": 20},
    {**basic_params, "num_ap": 150, "num_ue": 15, "tau_cf": 20},
    {**basic_params, "num_ap": 200, "num_ue": 20, "tau_cf": 20},
]

results = []

for par in all_params:
    cell_free_sim = CellFree(**par)
    rng = np.random.default_rng(12331)
    res = cell_free_sim.simulate(rng, 300, 100)

    label = (
        f"$M={par['num_ap']}$, "
        f"$K={par['num_ue']}$, "
```



```

        r"$\tau_{\text{cf}}$ " f"={par['tau_cf']}"$)
    results.append((label, res))

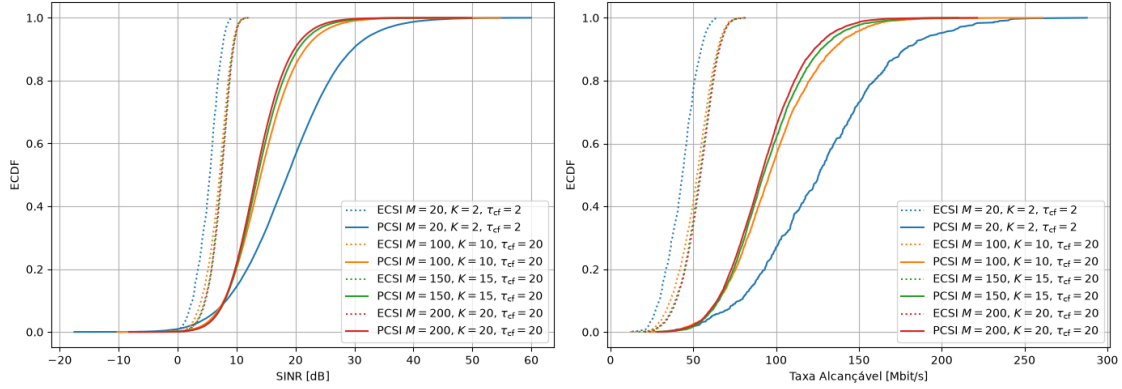
plot_scenarios(results)
plt.show()

```

```

100%|      | 300/300 [00:00<00:00, 338.98it/s]
100%|      | 300/300 [00:03<00:00, 77.85it/s]
100%|      | 300/300 [00:08<00:00, 37.27it/s]
100%|      | 300/300 [00:14<00:00, 20.07it/s]

```



Com esse caso, é possível notar que o fator mais relevante para manutenção de performance não é densidade de APs por espaço, mas sim a razão M/K . Para os cenários com $K \geq 10$, a performance de todos foi extremamente semelhante. Já o cenário com $K = 2$ apresentou um resultado consideravelmente diferente, por mais que a razão tenha sido mantida. A taxa alcançável com conhecimento estatístico do canal apresentou ligeira piora quando comparada com os outros cenários, mas com CSI perfeita apresentou melhora considerável. A piora ocorre devido a ausência de *channel hardening*, com a presença de poucos APs, enquanto a melhora ocorre devido à presença de baixíssima interferência no regime instantâneo após considerar inclusive as perdas de percurso.

3.5 Cenário 5 - Variando a Potência de Transmissão

Para o cenário base, foram utilizados diferentes potências de transmissão de dados, com $P_d \in \{0.1, 1, 10, 100\}$ mW.

```

[14]: all_params = [
    {**basic_params, "downlink_pow_db": 10*np.log10(0.0001)},
    {**basic_params, "downlink_pow_db": 10*np.log10(0.001)},
    {**basic_params, "downlink_pow_db": 10*np.log10(0.01)},
    {**basic_params, "downlink_pow_db": 10*np.log10(0.1)},
]

results = []

```

```

for par in all_params:
    cell_free_sim = CellFree(**par)
    rng = np.random.default_rng(12331)
    res = cell_free_sim.simulate(rng, 300, 100)

    power = 10**(par["downlink_pow_db"]/10)*1000
    label = f"$P_d={power}$ mW"

    results.append((label, res))

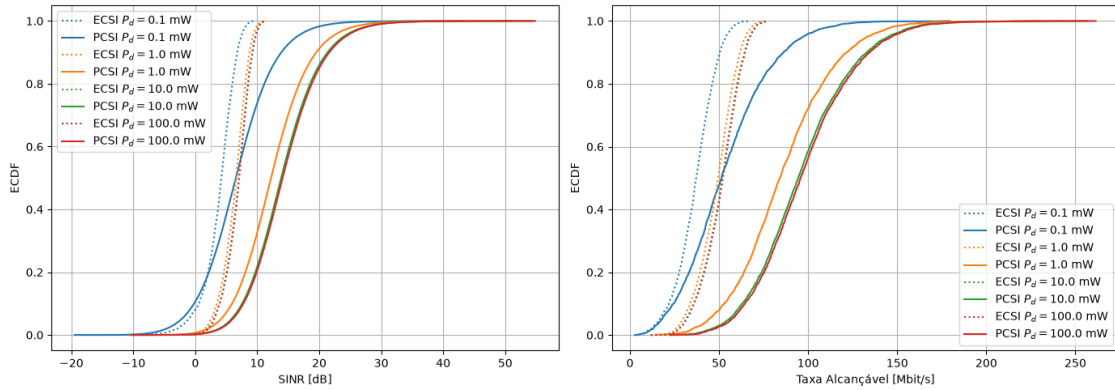
plot_scenarios(results)
plt.show()

```

```

100%|      | 300/300 [00:03<00:00, 80.86it/s]
100%|      | 300/300 [00:03<00:00, 83.58it/s]
100%|      | 300/300 [00:03<00:00, 85.11it/s]
100%|      | 300/300 [00:03<00:00, 81.54it/s]

```



Os resultados demonstram que, a partir de certa potência, aumentar a potência de transmissão de dados não auxilia na melhora de SINR e de taxa alcançável. Isso é esperado pois, tanto o sinal desejado quanto a interferência são multiplicados por um mesmo escalar derivado dessa potência. O que ocorre é que, quando $P_d \gg \sigma_w$, o canal passa a não depender do ruído, e aumentar essa potência não surte mais efeito.

3.6 Cenário 6 - Variando o Comprimento da Sequência de Pilotos

Para o cenário base, foram utilizados diferentes comprimentos de sequência de pilotos, com $\tau_{cf} \in \{10, 50, 100\}$.

```

[17]: all_params = [
    {**basic_params, "tau_cf": 10},
    {**basic_params, "tau_cf": 50},
    {**basic_params, "tau_cf": 100},

```

```

]

results = []

for i, par in enumerate(all_params):
    cell_free_sim = CellFree(**par)
    rng = np.random.default_rng(12331 + i)
    res = cell_free_sim.simulate(rng, 300, 100)

    label = rf"$\tau_{\text{cf}}$={par['tau_cf']}"
    results.append((label, res))

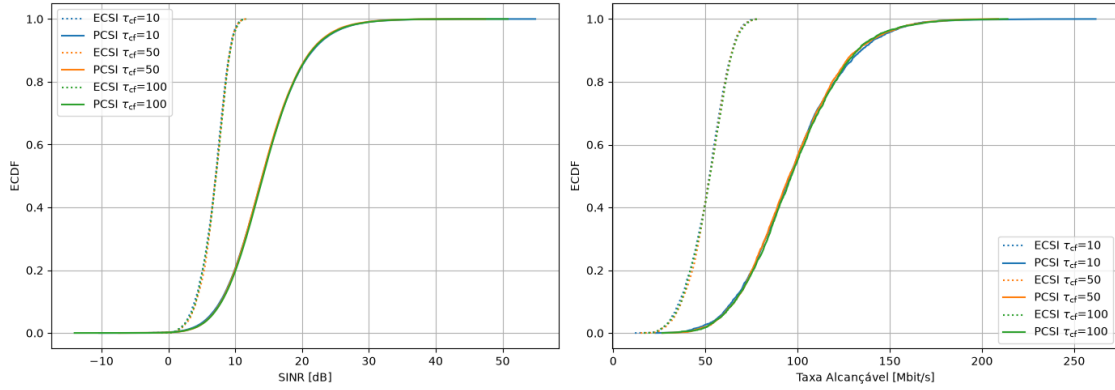
plot_scenarios(results)
plt.show()

```

```

100%|      | 300/300 [00:03<00:00, 79.77it/s]
100%|      | 300/300 [00:03<00:00, 83.34it/s]
100%|      | 300/300 [00:03<00:00, 82.87it/s]

```



É interessante que, alterando τ_{cf} no cenário original, todas as curvas se sobrepõem perfeitamente.

Considerando a estimação MMSE do canal, o sinal piloto recebido e projetado para o usuário k é dado por

$$y_{m,k} = \sqrt{\tau_{\text{cf}} P_p} g_{m,k} + w_{m,k}, \quad (2)$$

onde $w_{m,k}$ representa o ruído complexo gaussiano.

A estimativa do canal é obtida por

$$\hat{g}_{m,k} = c_{m,k} y_{m,k}, \quad (3)$$

com

$$c_{m,k} = \frac{\sqrt{\tau_{\text{cf}} P_p} \beta_{m,k}}{\tau_{\text{cf}} P_p \beta_{m,k} + \sigma_w^2}. \quad (4)$$

A variância da estimativa é

$$\gamma_{m,k} = \mathbb{E}\left\{|\hat{g}_{m,k}|^2\right\} = \sqrt{\tau_{\text{cf}} P_p} \beta_{m,k} c_{m,k}, \quad (5)$$

ou, substituindo a expressão de $c_{m,k}$,

$$\gamma_{m,k} = \frac{\tau_{\text{cf}} P_p \beta_{m,k}^2}{\tau_{\text{cf}} P_p \beta_{m,k} + \sigma_w^2}. \quad (6)$$

Observa-se que, quando

$$\tau_{\text{cf}} P_p \beta_{m,k} \gg \sigma_w^2, \quad (7)$$

tem-se

$$\gamma_{m,k} \approx \beta_{m,k}, \quad (8)$$

ou seja, a qualidade da estimação torna-se praticamente limitada apenas pelo ganho de larga escala do canal. Nesse regime, aumentar o comprimento da sequência de pilotos não produz melhorias perceptíveis na estimação e, consequentemente, na SINR e na taxa alcançável.

3.7 Cenário 7 - Variando a Potência dos Pilotos e Comprimento da Sequência de Pilotos

```
[ ]: all_params = [
    /**basic_params,
        "tau_cf": 10,
        "pilot_pow_db": 10*np.log10(0.0001)},

    /**basic_params,
        "tau_cf": 10,
        "pilot_pow_db": 10*np.log10(0.001)},

    /**basic_params,
        "tau_cf": 100,
        "pilot_pow_db": 10*np.log10(0.0001)},

    /**basic_params,
        "tau_cf": 100,
        "pilot_pow_db": 10*np.log10(0.001)},
]

results = []
```

```

for i, par in enumerate(all_params):
    cell_free_sim = CellFree(**par)
    rng = np.random.default_rng(12331+i)
    res = cell_free_sim.simulate(rng, 300, 100)

    power = 10**(par["pilot_pow_db"]/10)*1000
    label = rf"$\tau_{\text{{cf}}}={par['tau_cf']}$, $P_p={power}$ mW"

    results.append((label, res))

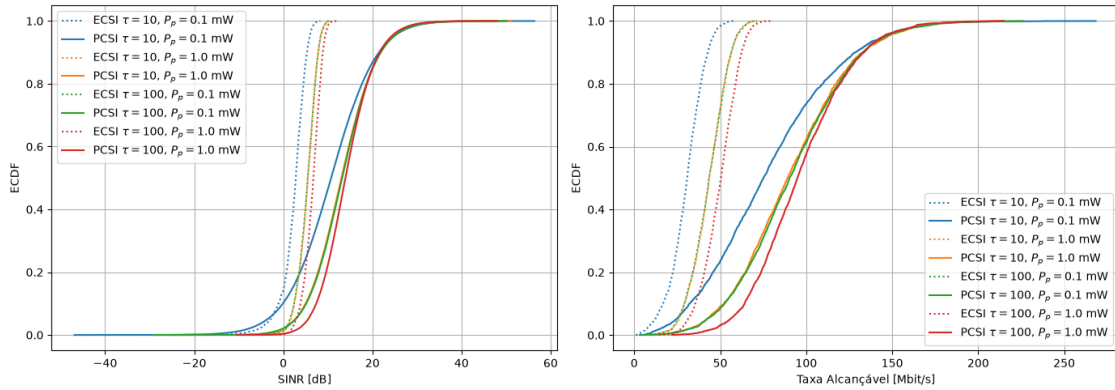
plot_scenarios(results)
plt.show()

```

```

100%|      | 300/300 [00:03<00:00, 80.10it/s]
100%|      | 300/300 [00:03<00:00, 81.78it/s]
100%|      | 300/300 [00:03<00:00, 82.15it/s]
100%|      | 300/300 [00:03<00:00, 81.28it/s]

```



4 Conclusões